

"DNS Cache Poisoning Like it's 2006" in USENIX'26 분석

이동훈

dhsama51@gmail.com

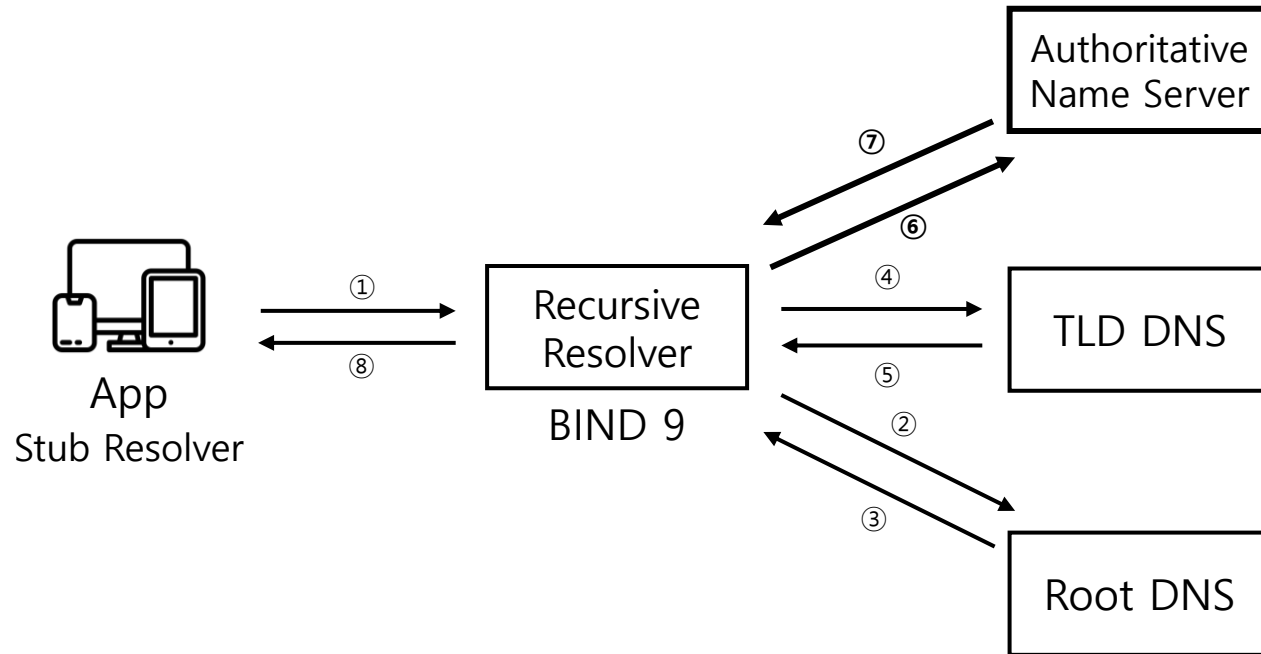
2026.08.24

Contents

1. Background - DNS Resolution & Packet Acceptance / Related Work
2. DNS Cache Poisoning Attack Structure
3. Threat Model
4. Breaking BIND PRNG, Xoshiro128**
5. Attack Variants - RRset / RRset-ANY / LotL / RR-QMA
6. Experiments and Evaluation
7. Existing Defense Mechanisms
8. Challenges
9. Contributions
10. Limitations
11. More Interested Papers (in IEEE S&P'26, USENIX'26)

1. Background - DNS Resolution & Packet Acceptance

- Typical DNS flow



⑥ www.target.example 주소 요청 query
Resolver IP: *port3* → *ANS:53* TXID= *txid4*
QNAME = www.target.example, QTYPE = A

⑦ Resolver에 www.target.example 주소 response
→ **DNS Cache**에 ANS, www.target.example 저장
ANS:53 → Resolver IP: *port3* TXID= *txid4*
Answer www.target.example A 203.0.113.10 TTL 300

- Resolver가 ③/④/⑦ accept하는 조건

1. Source IP = Root DNS / TLD DNS / ANS의 IP
2. **Destination Port = ②/④/⑥의 UDP source Port (16bit)**
3. **TXID 일치 (16bit)**
4. QR, QNAME, QTYPE 등 기타 field 유효
→ 공격 성공을 위해 **16bit port + 16bit TXID 예측**이 필요

1. Background - Related Work

- Related Work

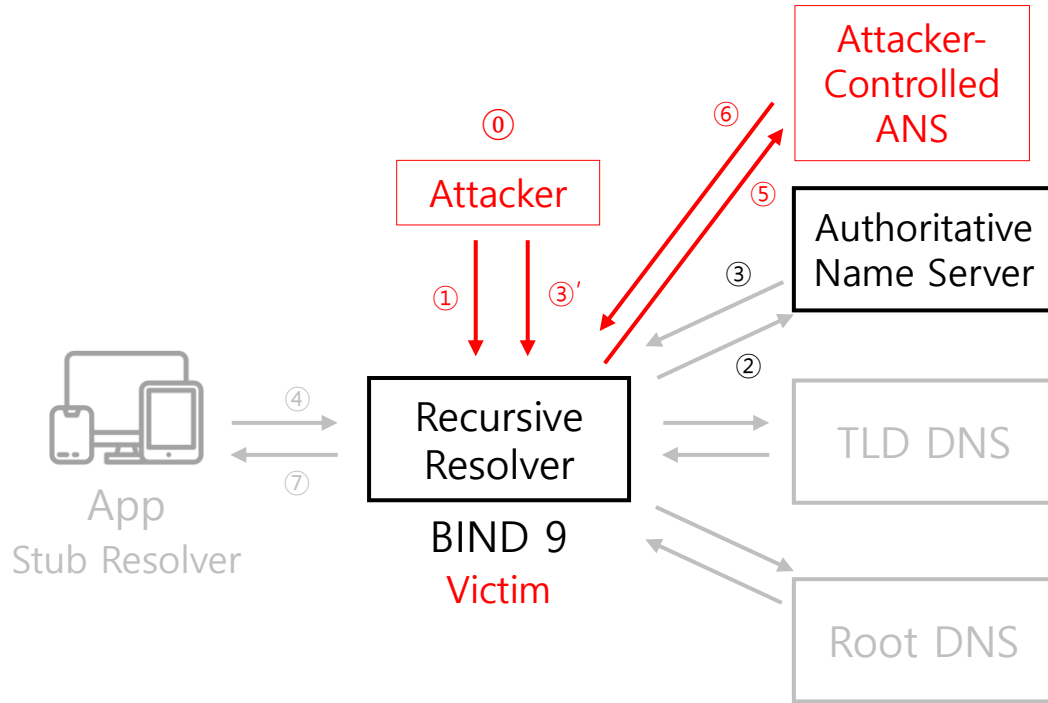
시기	대표 연구	공격 방법	spoofed packets	attacker ANS	방어 기법
~2006	Klein — BIND 8 / 9 PRNG	Randomized TXID 예측	few	불필요	X → PRNG 교체
2008	Kaminsky	TXID를 brute force해 NS Delegation	다수	필요	X → UDP source port randomization
2012~	Fragmentation (Herzberg-Shulman)	2nd fragment	—	필요	RFC 9715(drop fragmented DNS q/r)
2020 / 21	SADDNS (Man et al., CCS)	port only (ICMP side channel)	수만	필요	stateful FW(unexpected traffic)
2026	DNS Cache Poisoning Like it's 2006	Randomized TXID+port 동시 예측해 NS Delegation	few(3~4)	필요 / 불필요 기법 모두 존재	DNSSEC 제외한 기법 영향 없음

Majestic Million top 100의 **95%가 non-DNSSEC** → 현재 환경에서 충분히 유효

2006년 이후 PRNG 교체 및 UDP port도 randomize → 그러나 여전히 **few-packet** 공격이 가능 → "Like it's 2006"

2. DNS Cache Poisoning Attack Structure

- DNS Cache Poisoning Attack Structure



① **RRset 또는 RR-QMA Attack**으로 PRNG state x_0 를 사전에 계산
→ PRNG state로 DNS response의 **TXID**와 **UDP source port**를 예측

① **Trigger Query**: **www.target.example**에 대한 새 DNS Query 유도

Attacker:임시 포트	→ Resolver IP:53	TXID= <i>txid1</i>
QNAME=www.target.example		QTYPE=A

② **Resolver가 정상 ANS에 www.target.example 주소 질의**

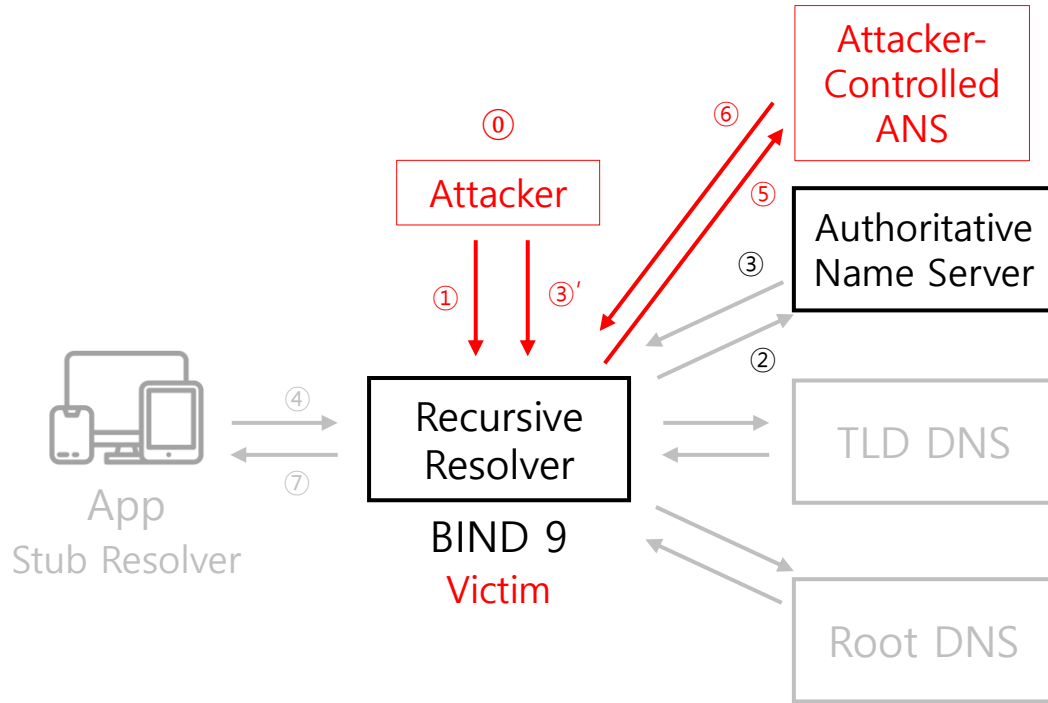
Resolver IP: <i>port1</i>	→ Legitimate ANS IP:53	TXID= <i>txid2</i>
QNAME=www.target.example		QTYPE=A

③' **정상 ANS보다 먼저 TXID, UDP port 위조 DNS Response 전송**

Legitimate ANS IP:53	→ Resolver IP: <i>port1</i>	TXID= txid2
Authority target.example NS ns-attacker.attacker.example		
Additional ns-attacker.attacker.example A Attacker-DNS-IP		
→ 위조된 NS Delegation을 DNS Cache에 저장		

2. DNS Cache Poisoning Attack Structure

- DNS Cache Poisoning Attack Structure



④ Device가 Resolver에게 `www.target.example` 주소 질의

device IP:임시 포트 → Resolver IP:53	TXID=txid3
QNAME=www.target.example	QTYPE=A

⑤ DNS Cache를 보고 `www.target.example` 질의를 **Attacker-Controlled DNS로 전달**

Resolver IP:port2 → Attacker-Controlled DNS IP:53	TXID=txid4
QNAME=www.target.example	QTYPE=A

⑥ Resolver에 `www.target.example`이 아닌 **www.attacker.example**의 주소 반환

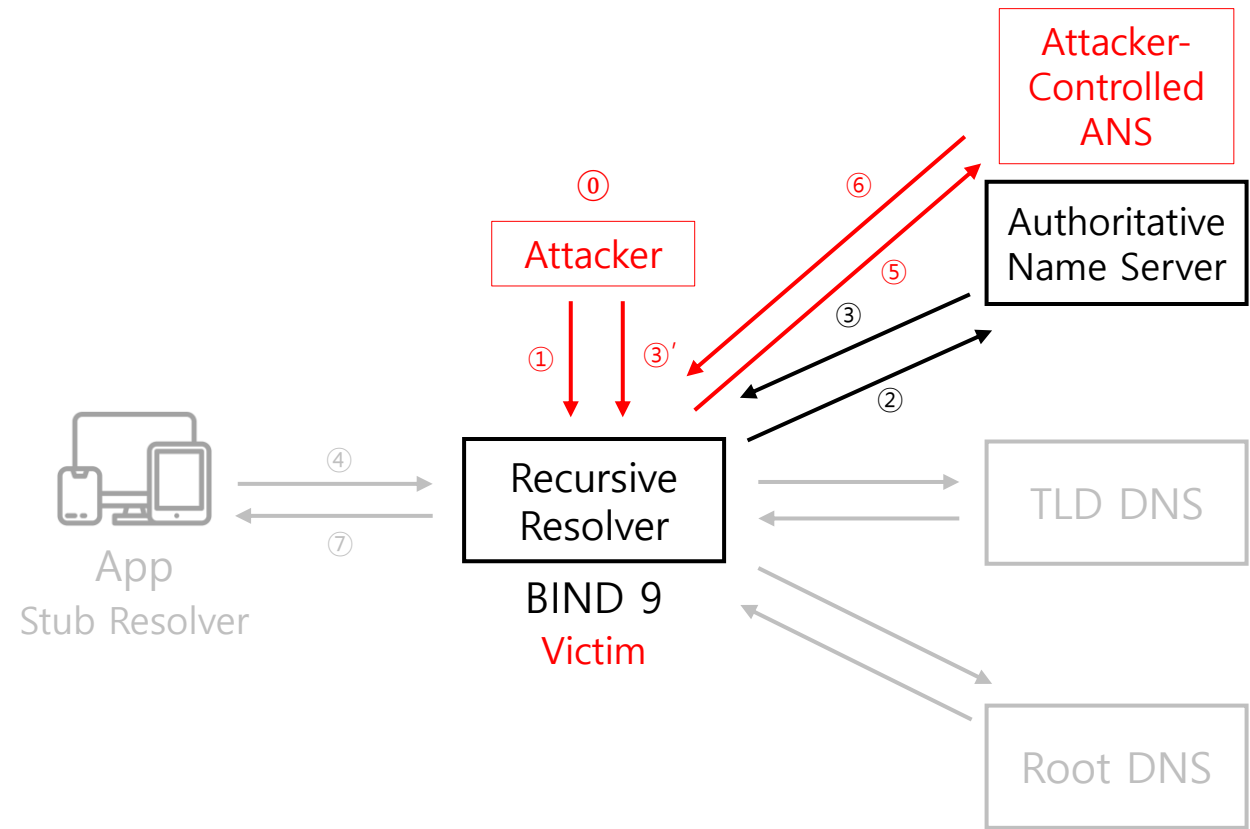
Resolver IP:53 → Device IP:임시 포트	TXID=txid4
Answer www.target.example A Attacker-Specified-IP	

⑦ Device에 `www.target.example`이 아닌 `www.attacker.example`의 주소 반환

Resolver IP:53 → Device IP:임시 포트	TXID=txid3
Answer www.target.example A Attacker-Specified-IP	

3. Threat Model

- DNS Cache Poisoning Attack Structure



Attacker

- 1. off-path: **cannot intercept** resolver-authoritative traffic
- 2. can **send spoofed DNS responses** to target resolver
- 3. can send queries directly to the resolver

Attacker's Goal

- 1. Recover the resolver's **PRNG state** from observed queries or responses
- 2. Predict both the **UDP source port and TXID**

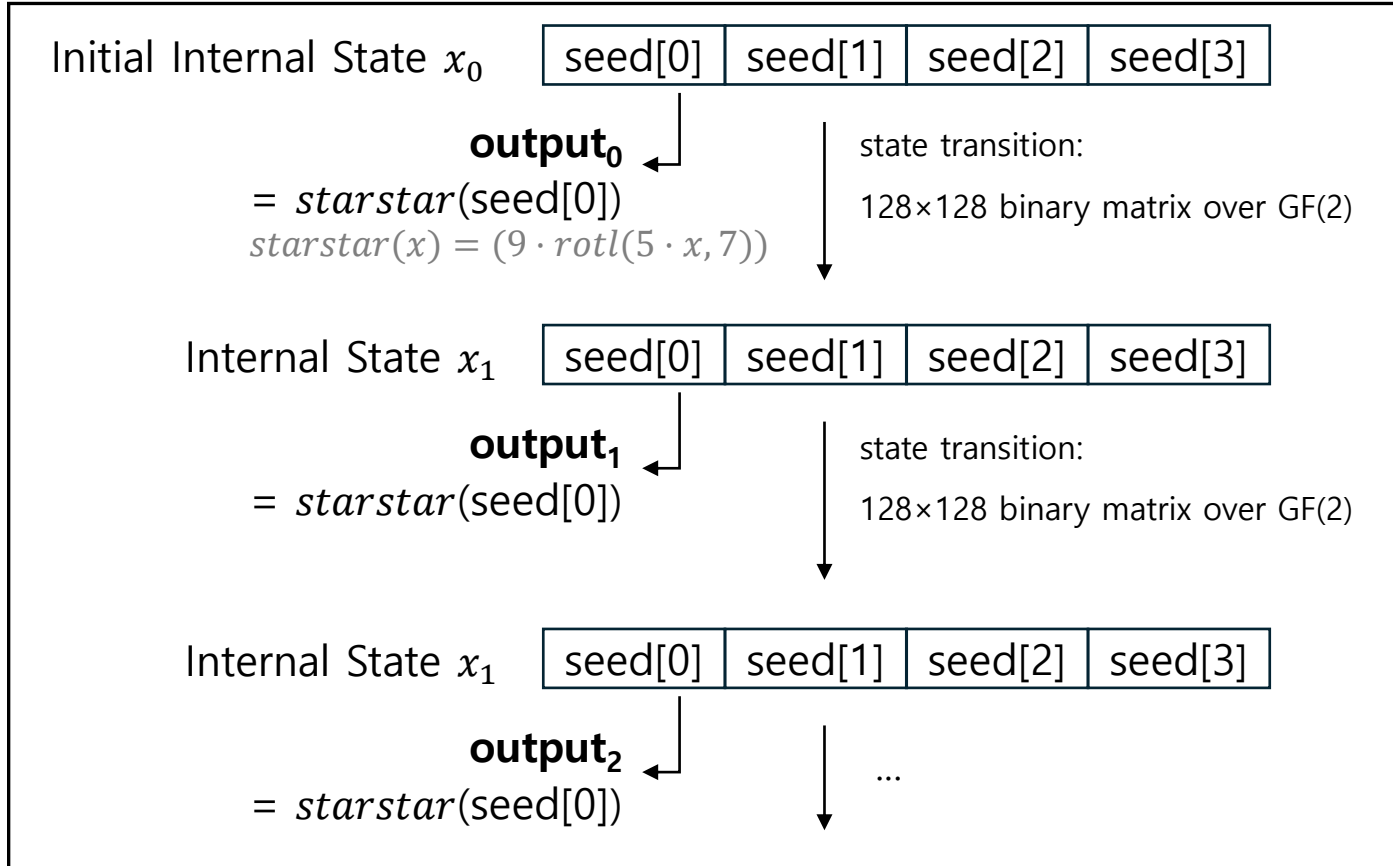
Assumption

- 1. target resolver: BIND 9 9.18/9.20/9.21 with defaults (RRset random, qmin, prefetch, DNS Cookies, **no DNSSEC**)
- 2. resolver load: ~10000 QPS

	9.18 resolver	9.20/9.21 resolver	9.20/9.21 forwarder
(Client side) RRset Attack (w/zone)	×	○	○
(Client side) RRset Attack (LotL)	×	○	○
(Server side) RR-QMA Attack (ANS)	○	○	×

4. Breaking BIND PRNG, Xoshiro128**

Xoshiro128**: BIND's Pseudo Random Number Generator



PRNG output로 생성하는 것

1. TXID & UDP source port

UDP port, TXID를 번갈아 가며 생성함
UDP port: port range에 맞게 조정

TXID: output의 LSB 16bit

→ TXID에 $\text{starstar}^{-1}()$ 를 적용하면
 x_n 의 `seed[0]` 9bit를 알아낼 수 있음

2. RRset order

RRset을 shuffle하는 데 사용

→ RRset의 크기 $N \geq 23$ 이면
 x_n 의 `seed[0]` 32bit를 알아낼 수 있음

각 state x_n 은 $x_n = A^n x_0$, 즉 linear transition으로 표현 가능함

x_0 를 알아낸다면 TXID, UDP port를 위조하여 DNS Cache Poisoning이 가능함

5. Attack Variants — RRset / RRset-ANY / RR-QMA

	RRset Attack	RRset-ANY Attack	RR-QMA Attack
관측 채널	RRset order 32 bit × 5 (질의 연속 5회 로 분산 수집)	RRset order 32 bit × 5 (ANY 응답 1개 에 atomic 수집)	outbound query의 TXID 9 bit × 20 (resolver가 attacker's ANS에 요청하도록 유도)
공격 위치	client-side	client-side	server-side
attacker ANS	불필요 (일반 ANS에 attacker zone만 업로드)	불필요 (일반 ANS에 attacker zone만 업로드)	필요 (outbound query 관측용)
대상	9.20 / 9.21 resolver, forwarder	9.20 / 9.21 resolver, forwarder	9.18 / 9.20 / 9.21 resolver
강점	RRset-order random이 default라 별도 조건 불필요	atomicity → load에 내성, RRset 제약 극복 10000 QPS에서도 94% 초과 success	RRset-order가 random이 아니어도 성립 9.18 대응의 유일한 수단
제약	질의 사이에 타 client query가 끼어들 수 있음	UDP payload 크기가 커지면 response가 잘림 → record 수·타입 조정 필요	forwarder 불가, atomicity를 보장하지 못해 9.18 @10,000 QPS에서 100% 실패

※ RRset(Resource Record set): 동일한 name과 type을 가진 DNS resource record들의 집합

LotL (Living off the Land) – RRset variant

이미 크기가 23 이상인 RRset을 가진 **third-party ANS**를 이용

Attacker는 domain 운영, ANS 운영, 일반 ANS로의 zone 업로드도 필요 없음 → 흔적 최소화

5. Attack Variants — RRset / RRset-ANY / RR-QMA

RRset-based Attack (client-side)

Resolver에 cache된 RRset에 대해, RRset randomize에 사용된 Xoshiro128**의 output을 복구함

ex. Fisher-Yates shuffle: [R0, R1, R2, R3, R4], output=8
x mod 5 = 3 → [R3, R1, R2, R0, R4]
x mod 4 = 0 → [R3, R1, R2, R0, R4]
x mod 3 = 2 → [R3, R1, R4, R0, R2]
x mod 2 = 0 → [R3, R1, R2, R0, R4]
x mod 1 = 0 → [R3, R1, R2, R0, R4]
→ 역으로 계산하면 CRT에 의해 $x \bmod 60 (= \text{lcm}(1, \dots, 5)) = 8$

$N \geq 23$ 이면 $\text{lcm}(1, \dots, N) > 2^{32}$ 이므로, $0 \leq x < 2^{32}$ 인 32bit PRNG output을 유일하게 결정할 수 있음

$\text{starstar}^{-1}(x)$ 으로 **PRNG state $x_t = A^t x_0$ 의 seed[0] 32bit**를 복구할 수 있음

$A^0 x_0, A^1 x_0, A^2 x_0, A^3 x_0, A^4 x_0$ 32bit씩 총 160bit로 x_0 128bit에 대한 선형 방정식

→ enough to solve using Gaussian elimination over GF(2)

5. Attack Variants — RRset / RRset-ANY / RR-QMA

RRset Attack (LotL variant) (client-side)

Attacker의 domain을 담당하는 ANS가 large RRset($N \geq 23$)을 준비하지 않고,
이미 존재하는 **third-party large RRset을 이용하여 RRset Attack**을 수행하는 방법

RRset-ANY Attack (client-side)

DNS ANY type query를 이용해 여러 RRset 순서를 atomic하게 추출하는 방법
반환된 각 RRset으로부터 **연속된 output을 얻을 수 있어** PRNG state를 안정적으로 복원할 수 있음

<RRset Attack>

공격자 query 1 → shuffled A RRset 반환

(다른 사용자의 query가 끼어들 수 있음)

공격자 query 2 → shuffled TXT RRset 반환

(다른 사용자의 query가 끼어들 수 있음)

공격자 query 3 → shuffled MX RRset 반환

(다른 사용자의 query가 끼어들 수 있음)

공격자 query 4 → shuffled MX RRset 반환

(다른 사용자의 query가 끼어들 수 있음)

공격자 query 5 → shuffled MX RRset 반환



<RRset-ANY Attack>

ANY type query → shuffled A RRset,
shuffled AAAA RRset
shuffled TXT RRset,
shuffled HTTPS RRset,
shuffled MX RRset을
하나의 DNS response에 넣어서 반환

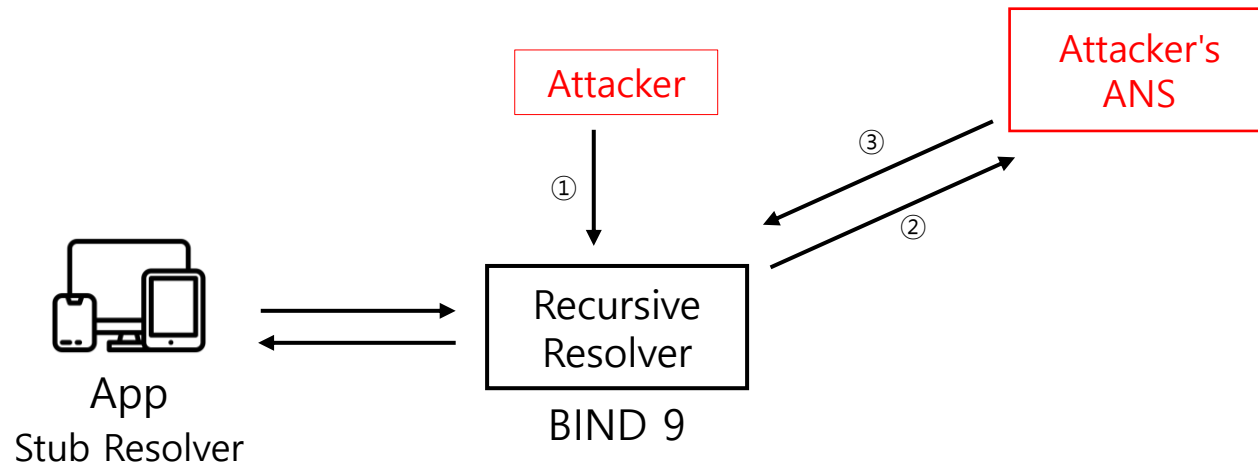
5. Attack Variants — RRset / RRset-ANY / RR-QMA

RR-QMA(Query-name Minimization Assisted) Attack (server-side)

Attacker가 multiple query를 보내서 Resolver가 Attacker의 ANS로 query를 보내게 만들고,
그 **query의 TXID를 관찰함**

TXID를 통해 **PRNG state $x_t = A^t x_0$ 의 seed[0] LSB 9bit**를 얻을 수 있고,
 $A^0 x_0, A^2 x_0, A^4 x_0, \dots, A^{38} x_0$ 9bit, 총 180bit로 x_0 128bit에 대한 선형 방정식
→ solve using Gaussian elimination over GF(2)

resolver의 **qmin**(default)은 minimizing query를 보내고
종속 쿼리를 queue에 대기시키다가 한번에 처리하여
near-atomicity를 달성하고 성공률을 높임



$$\begin{aligned} \text{TXID} &= \text{starstar}(\text{seed}[0]) \bmod 2^{16} \\ &= 9 \cdot \text{rotr}(5 \cdot \text{seed}[0], 7) \bmod 2^{16} \\ &= x \\ \Rightarrow x &= 9^{-1} \cdot \text{TXID} \bmod 2^{16} \\ x & \text{ (bit representation: 16 bits, MSB to LSB)} \\ & \text{MSB} \underbrace{\hspace{10em}}_{=y} \text{LSB} \\ y &= x \gg 7 = 5 \cdot \text{seed}[0] \bmod 2^9 \\ z &= 5^{-1} \cdot y \bmod 2^9 = \text{seed}[0] \bmod 2^9 \\ \therefore \text{seed}[0] \text{ LSB 9bit} &= 5^{-1}((9^{-1} \cdot \text{TXID}) \gg 7) \bmod 2^9 \end{aligned}$$

② **www.attacker.example** 미할당 타입 요청 질의
Resolver IP: **port3** → ANS:53 TXID= **txid4**
QNAME = **www.attacker.example**, QTYPE = 40000

6. Experiments and Evaluation

Variant	BIND Ver.	Protocol	Attack Success
RRset	9.20.11	UDP	100.00%
RRset	9.20.11	TCP	100.00%
RRset	9.21.10	UDP	100.00%
RRset	9.21.10	TCP	100.00%
RRset-ANY	9.20.11	UDP	100.00%
RRset-ANY	9.20.11	TCP	100.00%
RRset-ANY	9.21.10	UDP	100.00%
RRset-ANY	9.21.10	TCP	100.00%
RR-QMA	9.18.38	UDP	97.00%
RR-QMA	9.18.38	TCP	92.00%
RR-QMA	9.20.11	UDP	100.00%
RR-QMA	9.20.11	TCP	99.00%
RR-QMA	9.21.10	UDP	96.00%
RR-QMA	9.21.10	TCP	97.00%

Table 2: Summary of baseline attacks against latest (at the time of writing) BIND versions: 9.18, 9.20 and 9.21.

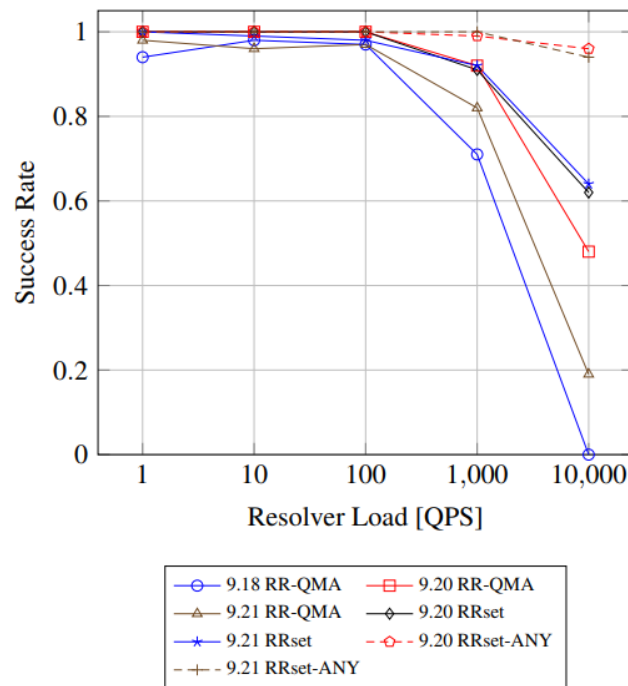


Figure 3: Attack success rate vs. resolver load.

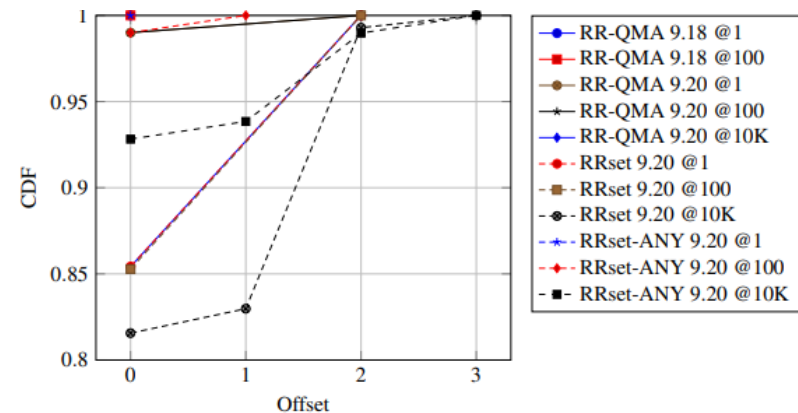


Figure 4: CDF of PRNG offset values per attack variant and load. Most attacks succeed with low offsets, even under heavy load. RR-QMA 9.18 @ 10K is not plotted since the attack fails 100% of the time. Note that RR-QMA 9.20 @ 1 graphically coincides with RR-QMA 9.20 @ 100, and RR-QMA 9.20 @ 10K graphically coincides RRset 9.20 @ 1 and RRset 9.20 @ 100.

RR-QMA는 qmin의 near-atomicity가 20개 queries에서 보장되지 않는 경우 공격 실패
 → Baseline에서도 100% succeed하지 못했고, **BIND 9.18 RR-QMA 10,000 QPS**에서는 100% 실패
 (BIND 9.18은 query 처리도 복잡)

7. Existing Defense Mechanisms

qmin

- DNS query의 불필요한 domain name 노출을 최소화하는 privacy 기술
- BIND 구현에서 minimized query와 관련된 query들을 모았다가 처리 → near-atomicity 제공해 공격을 보조함

DNS Cookies

- DNS Query/Response에 client/server cookie를 사용해 spoofing 방어
- BIND 구현에서 ANS의 cookie 지원 여부가 60초(9.18: 1800초) 후 만료되어 timeout window 발생
- 해당 window에서 spoofed response 10/10 success

Prefetching Records

- TTL 만료 직전 query 시 cache를 미리 갱신하는 기능
- A record가 아닌 NS delegation을 poisoning하므로, 갱신 query 자체가 Attacker's ANS로 향함

Anomaly Detection / stateful Firewall

- RRset 계열 최대 5 packet, RR-QMA 20 packet, 정상 query 형태
- UDP port probing, ICMP error 등 unexpected traffic 불필요 → Anomaly 탐지 어려움

8. Challenges

TXID와 UDP port 동시 예측 필요

- 선행 연구들은 TXID만 예측(Klein(2007), 당시 UDP port 고정)하거나 UDP port(Man et al.(2020), TXID는 brute force)만 예측

타 client의 query에 영향을 받지 않아야 함

- BIND의 response로부터 정보를 얻으므로 타 client의 query가 간접적으로 영향을 줄 수 있는 환경

DNS Cookies, DNSSEC, qmin, stateful Firewall 등의 기존 방어 기법을 극복해야 함

- 2006년과 달리, DNS 방어 기법도 꾸준히 연구되어 진보해왔음

BIND 9.18은 처리의 복잡성으로 공격 방법도 복잡함

- thread별로 PRNG를 따로 가지고, listener thread에서는 RRset order randomize만, worker thread에서는 TXID 생성만 담당함
→ RRset order로 PRNG state를 파악하더라도 TXID를 예측할 수 없음
- query name hash로 worker thread가 결정됨
→ 공격으로 알아낸 PRNG state가 쓰이지 않을 수 있음

9. Contributions

시장 점유율 1위인 BIND 9에 대해, unexpected traffic 없이 few spoofed packet으로 성공

- Anomaly Detection, stateful Firewall로도 탐지 어려움

client-side / server-side 2가지 공격이 모두 강력

- server-side(RR-QMA)는 BIND 9.18까지도 공격이 가능함
- client-side(RRset)은 RRset이라는 새로운 attack vector를 활용하며, attacker ANS가 불필요하고, LotL에서는 도메인 소유조차 불필요함
- 두 방법 모두 near-atomicity / atomicity를 갖도록 설계하여 ~10,000 QPS에서도 성공률이 높음 (9.18 RR-QMA 10,000 제외)

Xoshiro 및 PRNG 관련 취약점을 제시

- output의 LSB $k(\geq 8)$ bit 이상을 알고 있다면, seed[0]의 LSB $k-7$ bit를 계산할 수 있음
- BIND에서는 32bit로 shuffle → 경우의 수가 2^{32} 이므로 RRset order 가짓수 $N!$ 이 더 큰 경우 완전한 shuffle이 불가능

10. Limitations

자체 설치한 default configuration, open resolver 5곳만 평가

- ISP or 기업이 운영하는 환경에서는 효과가 감소할 수 있음

non-DNSSEC 환경, source IP spoofing 능력 가정

- Majestic Million top 100의 95%가 non-DNSSEC이지만, 우회가 불가능하다는 한계는 존재
- AS의 30.5%만 spoofed 패킷을 필터링하지 않음(CCS'21) → 다른 네트워크의 resolver 도달하기 전 필터링 가능성

BIND 9.18에 대한 공격 실패 가능성

- BIND 9.18은 RR-QMA만 가능하지만, 10000 QPS에서는 100% 실패 → 실제 환경에서는 공격이 어려울 수 있음

LotL → 크기가 23 이상인 RRset을 확보할 수 있는가?

- Tranco top 100만 domain 중 108개(0.0108%) 가 20개 초과 A record를 가짐(USENIX'25) → $N \geq 23$ RRset은 108개 이하
- third-party RRset을 어떻게 찾는가? → 100만 번 A query로 확인(USENIX'25) → 공격이 stealth하지 않을 수 있음

attacker가 정상 ANS보다 빨리 응답해야 함

- client-side / server-side에 모두 적용되는 조건
- attacker가 ANS보다 빠를 수 있음이 이론적으로 가능하지만, 실제 인터넷에서의 실험은 존재하지 않음

11. More Interested Papers (in IEEE S&P'26, USENIX'26)

1. 인증 프로토콜 설계 결함 이용 공격

Title	Conference	Content
Opossum Attack: Application Layer Desynchronization using Opportunistic TLS	USENIX'26	Implicit/Opportunistic TLS 동시 지원 시 desynchronization으로 TLS 인증 우회
Analyzing the WebRTC Ecosystem and Breaking Authentication in DTLS-SRTP	USENIX'26	DTLS-SRTP 인증 우회 취약점을 자동 탐지, 24개 실 서비스에서 검증
Audience Injection Attacks: A New Class of Attacks on Web-Based Authorization and Authentication Standards	IEEE S&P'26	OAuth/OIDC의 audience 검증 결함으로 계정 탈취 가능한 신규 공격 군

2. 프로토콜 레벨 자원 고갈 이용 DoS

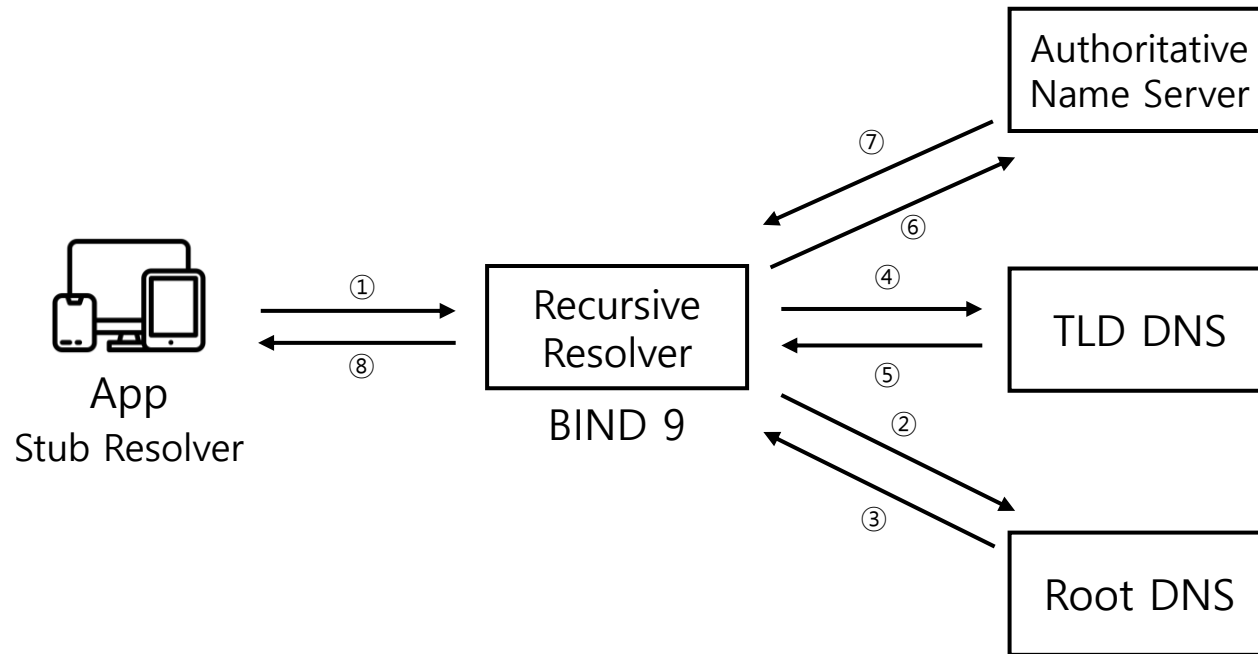
Title	Conference	Content
MadeYouReset: Exploiting HTTP/2 Server-Side Resets for Large-Scale DoS	IEEE S&P'26	HTTP/2 서버의 RST_STREAM이 백엔드 처리를 취소하지 못하는 결함 이용하는 DoS
TED: Abusing Tunnel Hosts and IPv6 Extension Headers for Pulsing DoS Attacks	USENIX'26	IPv6 터널의 Extension Header 처리 결함으로 지속 분산 트래픽을 지연시간 조작으로 특정 순간에 집중시켜 순간 폭증시킴
Bridges to Self: Silent Web-to-App Tracking on Mobile via Localhost	USENIX'26	Localhost/mDNS/WebRTC로 브라우저-앱 격리 우회, 웹-앱 계정 연결 트래킹

Thank you

Q&A

A. About DNS Cache Poisoning Attack

- Typical DNS flow



IP/UDP Header

src IP src port
dst IP dst port
protocol(UDP)

DNS Header

TXID(16bit) QR(0/1) RD(0/1: Recursion Desired)
QDCOUNT ANCOUNT NSCOUNT
ARCOUNT

DNS Message

Question Section

QNAME(Domain Name)
QTYPE(A, AAAA, NS, ...)
QCLASS(IN)

Answer Section

질문에 대한 직접적인 답변 ex. www.example.com A 203.0.113.10 TTL 300

Authority Section

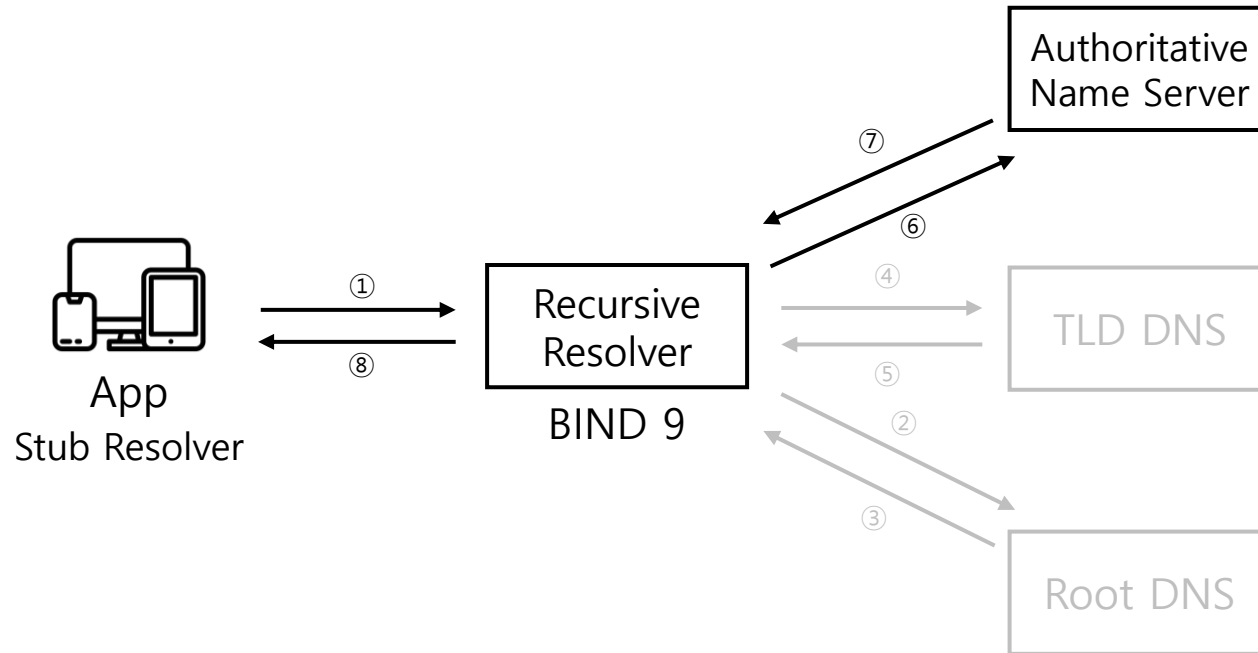
도메인의 관리자 정보 ex. .com NS a.gtld-servers.net

Additional Section

NS 이름의 IP 등의 정보 ex. a.gtld-servers.net A ...

A. About DNS Cache Poisoning Attack

- Typical DNS flow



① **www.target.example** 주소 요청

device IP:임시 포트 → Resolver IP:53 TXID=*txid1* QR=0(Query)
QNAME=www.target.example QTYPE=A

② **.example TLD DNS** 주소 질의

Resolver IP:port1 → Root DNS IP:53 TXID=*txid2* QR=0(Query)
QNAME=example, QTYPE=NS

③ **.example TLD DNS** 주소 반환

Root DNS IP:53 → Resolver IP:port1 TXID=*txid2* QR=1(Response)
Authority .example NS a.gtld-servers.net
.example NS b.gtld-servers.net ...
Additional a.gtld-servers.net A ...
b.gtld-servers.net A

④ **target.example**을 알려줄 **DNS** 주소 질의

Resolver IP:port2 → TLD DNS IP:53 TXID=*txid3* QR=0(Query)
QNAME=target.example, QTYPE=NS

⑤ **target.example**을 알려줄 **ANS** 주소 반환

TLD DNS IP:53 → Resolver IP:port2 TXID=*txid3* QR=1(Response)
Authority example.com NS ns1.example.com
example.com NS ns2.example.com
Additional ns1.example.com A 192.0.2.53
ns2.example.com A 192.0.2.54

⑥ **www.target.example** 주소 요청 질의

Resolver IP:port3 → ANS:53 TXID=*txid4* QR=0(Query)
QNAME = www.target.example, QTYPE = A

⑦ **Resolver**에 **www.target.example** 주소 반환+**DNS Cache**에 **ANS, www.target.example** 저장

ANS:53 → Resolver IP:port3 TXID=*txid4* QR=1(Response)
Answer www.target.example A 203.0.113.10 TTL 300

⑧ **Device**에 **www.target.example** 주소 반환

Resolver IP:53 → device IP:임시 포트 TXID=*txid1* QR=1(Response)
Answer www.target.example A 203.0.113.10 TTL 300